Digital Egypt Pioneers Initiative (DEPI) – Phase 5

E-commerce Data Integration & Analytics Pipeline

Project Documentation

Parts 1–3: Project Planning & Management · Literature Review · Requirements Gathering

Item	Details
Track	Data Engineering
Team name	Pipeline Pioneers
Team member 1	محمود سيد محمد علي
Team member 2	حسام عبدالمحسن صالح نصر
Team member 3	عبدالرحمن محمود عبدالمنعم مشعل
Team member 4	احمد جمال عزقلاني عطيه
Team member 5	احمد اسماعيل اسماعيل خليل
Technical Instructor	Mohamed Shehab
GitHub repository	https://github.com/aiahmedismail80-code/ecommerce-data-pipeline
Version / date	1.0 – October 2026

1. Project Planning & Management

1.1 Project Proposal

Overview. The E-commerce Data Integration & Analytics Pipeline collects, cleans, transforms and stores e-commerce data from several related files in a central data warehouse, and presents it in Power BI dashboards covering sales, customers, products and delivery. Python and Pandas handle ingestion and preparation, SQL Server stores the data, dbt transforms and tests it, and Power BI presents it. The initial source is the Brazilian E-Commerce Public Dataset by Olist.

Problem. E-commerce data is spread across separate files for orders, customers, products, payments and deliveries. Combining it by hand causes duplicates, inconsistent formats, missing values and late reporting. This project builds a repeatable pipeline that integrates the data, improves its quality and prepares it for reliable analytics.

Objectives

1. Integrate multiple related e-commerce datasets into one data platform.
2. Automate ingestion and preparation with Python.
3. Clean and validate the data for accuracy and consistency.
4. Design a dimensional data warehouse (fact and dimension tables).
5. Build SQL and dbt transformations with data quality tests.
6. Develop Power BI dashboards for sales, customers, products and delivery.
7. Keep code and documentation on GitHub.

Scope

In scope	Out of scope
Importing e-commerce data from CSV files	Building an e-commerce website
Profiling, cleaning and validating data with Python and Pandas	Processing real customer payments
Loading data into SQL Server staging tables	Accessing private commercial databases
Dimensional warehouse with fact and dimension tables	Production-scale cloud deployment
SQL and dbt transformations and data quality tests	Real-time streaming or machine learning
Power BI dashboards and KPIs	
Documentation and GitHub collaboration	

Technology stack. Data source: Olist dataset (CSV). Ingestion and preparation: Python and Pandas. Warehouse: SQL Server. Transformation and tests: SQL and dbt. Visualization: Power BI. Version control: Git and GitHub. The team confirms the final stack before implementation.

Architecture. The pipeline moves data through these stages: source CSV files → staging tables in SQL Server → cleaning and transformation (Python, SQL, dbt) → warehouse fact and dimension tables → Power BI dashboards. Data quality tests run at each stage and their results are recorded.

1.2 Project Plan

The schedule follows the DEPI deliverable dates in the Project Documentation Guidelines. Documentation parts 1–3 are due on 19 October 2026.

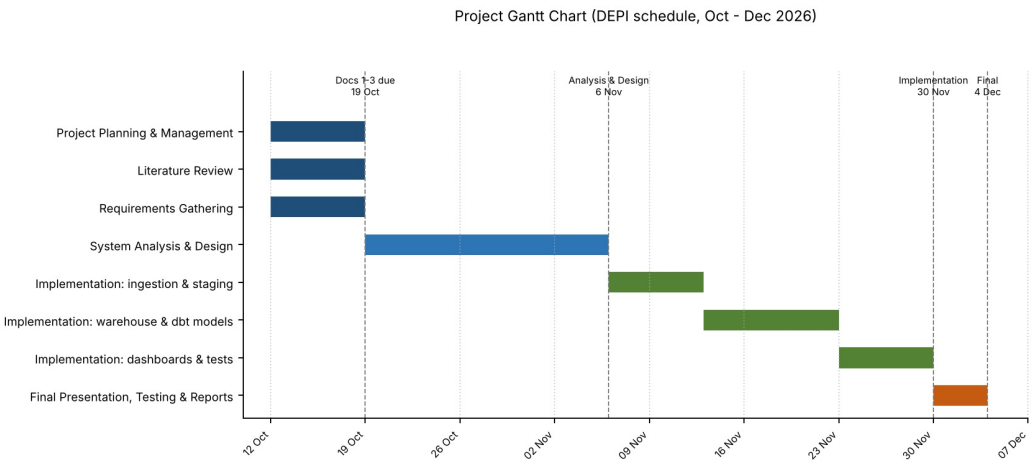


Figure 1: Project Gantt chart with milestones.

Milestone	Date	Deliverables
Planning & documentation	19 Oct 2026	Project plan, literature review, requirements
System Analysis & Design	6 Nov 2026	Use cases, architecture, data model (ERD), data flow diagram
Implementation	30 Nov 2026	Ingestion scripts, warehouse tables, dbt models and tests, Power BI dashboards, README
Final Presentation, Testing & Reports	4 Dec 2026	Test results, user manual, technical documentation, presentation, infographic

Resources. Five team members; the Olist public dataset; Python, Pandas, SQL Server, dbt, Power BI, Git and GitHub; the team’s own computers; and the supervision of the Technical Instructor, Mohamed Shehab.

1.3 Task Assignment & Roles

Each member owns one main technical area and supports the others. Every member must contribute to the implementation and understand the whole pipeline, because the Technical Instructor will ask each member about their contribution.

Member	Role	Responsibilities
محمود سيد محمد علي	Team leader; SQL Server staging; documentation	Coordinate meetings, track progress, communicate with the instructor; create the SQL Server staging tables and load the ingested files into them; maintain documentation, test cases and setup instructions
حسام عبدالمحسن صالح نصر	Data ingestion developer	Write the Python scripts that read the Olist CSV files, check their structure and row counts, and prepare them for loading; handle logging and errors
عبدالرحمن محمود عبدالمنعم مشعل	Data cleaning developer	Profile, clean and validate data with Pandas and SQL; document the cleaning rules
احمد جمال عزقلاني عطيه	Data warehouse developer	Design the warehouse schema (fact and dimension tables, keys and relationships); create the SQL Server warehouse tables and load the cleaned data into them
احمد اسماعيل اسماعيل خليل	dbt and BI developer	Build dbt models and data quality tests; analyze the warehouse data, define the KPIs, and build the Power BI dashboards

1.4 Risk Assessment & Mitigation Plan

Risk	Impact	Mitigation
Missing or inconsistent data	Incorrect results	Profile data, apply validation rules, document cleaning steps
Incorrect joins	Duplicate records, wrong totals	Define relationships and validate join cardinality
Development delays	Incomplete deliverables	Split work into milestones and review progress weekly
Uneven team participation	Lower quality, difficulty in the defense	Assign clear roles and track individual contributions
Incorrect dashboard metrics	Misleading insights	Check KPI definitions against SQL queries and source data

1.5 KPIs (Key Performance Indicators)

KPI	Measure
Data completeness	Share of required records with complete mandatory fields
Data validity	Share of records that pass the validation rules
Referential integrity	Required foreign keys are valid after cleaning
Pipeline reliability	The pipeline completes successfully on the test dataset
Pipeline run time	A full run finishes within an agreed time on the team’s hardware
Reconciliation accuracy	Record counts and totals match the source, apart from documented exclusions
Dashboard accuracy	Dashboard KPIs match validated SQL results

The team sets numerical targets after profiling the dataset.

2. Literature Review

2.1 Review of Similar Solutions

This section reviews datasets, architectures and approaches relevant to e-commerce data integration and explains how the project applies them.

Solution 1: Olist Brazilian E-Commerce Public Dataset. Source: Kaggle. About 100,000 orders from 2016 to 2018, with related tables for customers, orders, items, products, sellers, payments and reviews.

- *Advantages:* realistic, anonymized data; several related tables suited to integration; supports sales, customer, payment and delivery analysis; needs no private database.
- *Limitations:* historical data in downloadable files, so it does not show live source integration; needs preparation and relationship validation.
- *Relevance:* it is the project's initial source. The team follows the license terms shown on the dataset's Kaggle page.

Solution 2: Microsoft data warehousing and analytics architecture. Source: Microsoft Learn. It integrates several systems, stages data in a data lake, transforms it in a warehouse, and serves it through Power BI.

- *Advantages:* clear separation of ingestion, storage, transformation and reporting; repeatable loading.
- *Limitations:* relies on several Azure services, so it adds cost and is too complex for a small academic project.
- *Relevance:* the project follows the same principles with a simpler stack: Python, Pandas, SQL Server, dbt and Power BI.

Solution 3: Traditional e-commerce dashboard projects. These projects use prepared files, SQL queries or BI tools to analyze sales and customers.

- *Advantages:* clear business insights through charts, filters and KPIs.
- *Limitations:* they often focus on visualization only, with manual cleaning and little documentation of the transformation and testing steps.
- *Relevance:* this project adds a documented ingestion and transformation workflow, a dimensional warehouse and data quality tests. Power BI is the final layer, not the whole solution.

Comparison

Feature	Olist dataset	Microsoft architecture	Proposed project
Purpose	Provides data	Enterprise data integration	End-to-end academic pipeline
Data warehouse	Not provided	Enterprise services	SQL Server dimensional model
Data quality testing	Must be implemented	Depends on implementation	Validation and dbt tests
Visualization	Not the focus	Power BI	Power BI dashboards

Contribution. The reviewed resources each cover only part of the picture: a dataset, a reference architecture or an analytical approach. This project joins them in one documented, testable workflow. It is an educational proof of concept and does not claim a new data engineering method.

2.2 Feedback & Evaluation, Suggested Improvements, Final Grading Criteria

- **Feedback & Evaluation:** reserved for the lecturer's assessment, to be completed after the Technical Instructor's review.
- **Suggested Improvements:** incremental loading of new records only; automatic scheduling and alerts; an automatic data quality report after each run; a cloud-hosted warehouse.
- **Final Grading Criteria:** the marks for documentation, implementation, testing and presentation are set by DEPI and the Technical Instructor, and will be added once confirmed.

2.3 References

1. Olist. Brazilian E-Commerce Public Dataset by Olist. Kaggle. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
2. Microsoft. Data Warehousing and Analytics. Microsoft Learn. https://learn.microsoft.com/en-us/azure/architecture/example-scenario/data/data-warehouse

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Need
Business managers	Reliable information on sales, revenue, orders and performance
Sales analysts	Sales trends across time periods and product categories
Marketing analysts	Customer and product insights for marketing decisions
Logistics analysts	Order status, delivery performance and delays
Data engineers	A repeatable, maintainable process for ingestion, cleaning, transformation and loading
Project team and Technical Instructor	A working, documented, testable project that shows the required skills

3.2 User Stories & Use Cases

User stories

ID	User story
US-01	As a business manager, I want to see total sales, total orders and average order value, so that I can track performance.
US-02	As a sales analyst, I want to compare sales by month and product category, so that I can find trends.
US-03	As a logistics analyst, I want to see order status and average delivery time, so that I can spot delays.
US-04	As a data engineer, I want to run the whole pipeline with documented steps and see test results, so that results are repeatable and bad data is caught early.

Use cases

Use case	Actor	Description
UC-01 Load and clean data	Data engineer	Run the Python scripts to load the CSV files into staging tables, then clean and validate the data
UC-02 Build and test the warehouse	Data engineer	Run SQL and dbt models to create fact and dimension tables, then run the data quality tests
UC-03 View dashboards	Managers, analysts	Open the Power BI dashboards, apply filters and read the KPIs

3.3 Functional Requirements

ID	Requirement
FR-01	The system shall read the selected source files and load them into SQL Server staging tables
FR-02	The system shall combine related datasets using documented keys and relationships
FR-03	The system shall handle missing values, duplicates, inconsistent types and invalid records by defined rules
FR-04	The system shall validate required fields, unique identifiers and relationships before analytics use
FR-05	The system shall organize analytical data into fact and dimension tables using SQL and dbt models
FR-06	The system shall run uniqueness, not-null, accepted-values and relationship tests
FR-07	The system shall provide Power BI dashboards with sales, customer, product and delivery metrics: total sales, total orders, average order value, orders by status, sales by category, average delivery time and review score
FR-08	The system shall record execution results and give useful information on failures
FR-09	The project shall keep code, SQL models and documentation in one GitHub repository, with setup instructions

3.4 Non-functional Requirements

ID	Requirement	Description
NFR-01	Data quality	Apply documented validation rules and report the issues found
NFR-02	Reliability	Give consistent results when run repeatedly on unchanged input
NFR-03	Performance	Process the dataset in reasonable time on the team’s hardware
NFR-04	Maintainability	Organize code and transformations into modular, understandable parts
NFR-05	Security	Use public data and keep credentials out of GitHub
NFR-06	Usability	Use clear labels, consistent measures and suitable filters in dashboards

Assumptions and constraints. The public dataset is available in a usable format and contains enough fields for the planned analysis. The dataset is historical, not a live system, and implementation time and team hardware are limited. Cloud deployment and real-time processing are outside the scope unless the team adds them.

Acceptance criteria. The project is ready for evaluation when the data loads through the documented process, the cleaning rules and warehouse tables are documented, the data quality tests run with recorded results, the main dashboard figures match validated SQL results, the repository contains the code and documentation, and each member can explain their own contribution.